

Vybraná témata z algoritmů

Abstrakt

Poznámky k semináři VYTAL v letním semestru 2026. Poslední úprava 24. dubna 2026.

1 První seminář

Obsah. *formule výrokové logiky a booleovské obvody reprezentují (všechny) booleovské funkce; pravdivost, splnitelnost, sémantická ekvivalence, ekvivalence vzhledem ke splnitelnosti; Tseitsinův algoritmus.*

~

def. *n-ární booleovská funkce* je funkce $f : \{0, 1\}^n \rightarrow \{0, 1\}$. (0 je nepravda, 1 je pravda).

Otázka. *Kolik je n-árních booleovských funkcí?*

Došli jsme k tomu, booleovskou funkci f můžeme reprezentovat řetězcem 2^n bitů, ve kterém je i -tý bit je roven $f(b_1, b_2, \dots, b_n)$, kde $(b_1 b_2 \dots b_n)_2 = i$. (Nejdříve si můžeme přestavit tabulku s $n+1$ sloupci, kde v prvních n sloupcích jsou hodnoty jednotlivých argumentů $b_1, b_2, \dots, b_n$ a v posledním sloupci je hodnota $f(b_1, b_2, \dots, b_n)$. Pokud řádky uspořádáme tak, že číslo $(b_1 b_2 \dots b_n)_2$ roste, jdeme-li tabulkou dolů, tvoří obsah posledního sloupce žádaný řetězec.) n -ární booleovských funkcí je tak tolik, kolik je 2^n -bitových řetězců, tedy 2^{2^n} . $\square$

Příklady booleovských funkcí jsou funkce logických spojek, zejména CON (konjunkce), DIS (disjunkce), NEG (negace).

V dalším použijeme množinu proměnných $\text{VAR} = \{x_1, x_2, x_3 \dots\}$, některé z nich budeme občas značit i jinými symboly, např. y, z s případným indexem.

def. *Formule výrokové logiky* je řetězec nad abecedou tvořenou proměnnými, symboly pro logické spojky $\wedge, \vee, \neg$, a závorkami, který je vytvořen podle následujících pravidel.

- Proměnná je formule.
- Jsou-li F, G formule, potom jsou i $(F \wedge G)$, $(F \vee G)$, $\neg F$ formule.

Některé závorky můžeme při zápisu vynechávat (např. vnější závorky, případně místo $(F \wedge G) \wedge H$ psát $F \wedge G \wedge H$, podobně pro disjunkci). Základní definici můžeme rozšířit o symboly dalších spojek zavedením *zkratek*:

- $F \rightarrow G$ je zkratkou $\neg F \vee G$,
- $F \leftrightarrow G$ je zkratkou $(F \rightarrow G) \wedge (G \rightarrow F)$.

Množinu proměnných obsažených ve formuli F označíme $\text{VAR}(F)$.

def. *Ohodnocení proměnných* je částečné zobrazení $\alpha : \text{VAR} \rightarrow \{0, 1\}$. Množinu proměnných, které α ohodnocuje, označíme $\text{VAR}(\alpha)$.

def. Pokud $\text{VAR}(F) \subseteq \text{VAR}(\alpha)$ můžeme hovořit o pravdivosti $F[\alpha]$ formule F . Je definována následovně:

- pro proměnnou x máme $x[\alpha] = \alpha(x)$,
- $(F \wedge G)[\alpha] = \text{CON}(F[\alpha], G[\alpha])$,
- $(F \vee G)[\alpha] = \text{DIS}(F[\alpha], G[\alpha])$,
- $(\neg F)[\alpha] = \text{NEG}(F[\alpha])$.

def. Formule F s proměnnými $\{x_1, \dots, x_n\}$ *reprezentuje* n -ární booleovskou funkci f , pokud pro každé ohodnocení α ohodnocující všechny tyto proměnné máme $F[\alpha] = f(\alpha(x_1), \alpha(x_2), \dots, \alpha(x_n))$.

Otázka. *Existuje ke každé n -ární booleovské funkci formule, která ji reprezentuje?*

Tady jsme si připoměli konstrukci formule v normální formě (konjunktivní nebo disjunktivní). Konstrukci provádíme na základě pravdivostní tabulky. Pravdivostní tabulka, ze které vynecháme všechny sloupce mimo ohodnocení proměnných a pravdivosti dané formule je vlastně zápis booleovské funkce pomocí tabulky (a existuje pro každou booleovskou funkci). Podrobnosti konstrukce lze nalézt v kurzu diskretních struktur. Odpověď je tedy *ano*, navíc umíme potřebnou formuli explicitně zkonstruovat. $\square$

def. *n -ární booleovský obvod* C je orientovaný graf bez cyklů. Vrcholům říkáme brány, vede-li z brány g hrana do brány h , je h *vstupem* g . Dále platí následující podmínky.

- Graf obsahuje n bran bez vstupů, každé je přiřazena jiná proměnná z množiny $\{x_1, x_2, \dots, x_n\}$. Jsou to tzv. *vstupní* brány. (Pak $\text{VAR}(C) = \{x_1, x_2, \dots, x_n\}$)
- Brány s jedním vstupem jsou $\neg$ -brány,
- Brány se dvěma vstupy jsou $\wedge$ -brány nebo $\vee$ -brány.
- V grafu nejsou brány s více než 2 vstupy.
- Jedna brána je navíc označena jako *výstupní*.

Nyní definujeme pravdivost obvodu pro ohodnocení α takové, že $\{x_1, \dots, x_n\} \subseteq \text{VAR}(\alpha)$. Nejdříve definujeme pravdivost $g[\alpha]$ brány g .

- Pokud je g vstupní brána s proměnou x , pak $g[\alpha] = \alpha(x)$.
- Pokud je g $\neg$ -brána se vstupem h , pak $g[\alpha] = \text{NEG}(h[\alpha])$.
- Pokud je g $\wedge$ -brána se vstupy h_1, h_2 , pak $g[\alpha] = \text{CON}(h_1[\alpha], h_2[\alpha])$.
- Pokud je g $\vee$ -brána se vstupy h_1, h_2 , pak $g[\alpha] = \text{DIS}(h_1[\alpha], h_2[\alpha])$.

Pravdivost obvodu je rovna pravdivosti jeho výstupní brány. Analogicky formulím výrokové logiky můžeme definovat situaci, kdy obvod C reprezentuje booleovskou funkci.

def. Nechť G je booleovský obvod nebo formule výrokové logiky. Pokud existuje ohodnocení α tak, že $\text{VAR}(\alpha) \subseteq \text{VAR}(G)$ a $G[\alpha] = 1$, řekneme, že objekt G je *splnitelný*.

def. Nechť G, F jsou booleovský obvod nebo formule výrokové logiky (mohou to být objekty různého typu). Potom

- G a F jsou *sémanticky ekvivalentní*, pokud pro každé ohodnocení α takové, že $\text{VARS}(G) \cup \text{VARS}(F) \subseteq \text{VARS}(\alpha)$, platí $G[\alpha] = F[\alpha]$.
- G a F jsou *ekvivalentní vzhledem ke splnitelnosti*, pokud je G splnitelná právě když je F splnitelná.

Otázka. Existuje ke každé booleovské funkci booleovský obvod, který ji reprezentuje?

Odpověď je ano. Protože ke každé booleovské funkci existuje formule výrokové logiky, která ji reprezentuje, stačí najít postup transformace formule výrokové logiky na sémanticky ekvivalentní booleovský obvod. Tento postup jsme na semináři našli. $\square$

def. *Literál* je proměnná, nebo její její negace. *Klausule* je disjunkcí literálů. Formule je *konjunktivní normální formě* (CNF), pokud je konjunkcí klausulí.

Tseitsinův algoritmus. Pomocí Tseitsinova algoritmu můžeme transformovat obvod C na formuli F v CNF tak, že C a F jsou ekvivalentní vzhledem ke splnitelnosti. Protože libovolnou formuli umíme převést na jí sémanticky ekvivalentní obvod (viz předchozí otázka), lze provedením Tseitsinova algoritmu na takto získaný obvod převést libovolnou formuli na formuli v CNF, která je s ní ekvivalentní vzhledem ke splnitelnosti. Následují detaily Tseitsinova algoritmu.

Vstupem algoritmu je obvod C , výstupem je formule F . Proměnné obvodu C jsou $x_1, \dots, x_n$ a jeho brány jsou $g_1, \dots, g_m$. Proměnné ve formuli F jsou $x_1, \dots, x_n, g_1, \dots, g_m$. Ve formuli tedy máme jednu proměnnou pro každou proměnnou v C a jednu proměnnou pro každou bránu v C : značíme je stejně, v následujícím bude vždy jasné, kdy se jedná o proměnnou a kdy o bránu.

Algoritmus funguje tak, že projde všechny brány obvodu C a pro každou vytvoří formuli v CNF. Výsledná formule je potom jejich konjunktí. Označme aktuální bránu g , a její případné vstupy označme h_1, h_2 (g, h_1, h_2 tedy patří do množiny $\{g_1, \dots, g_m\}$).

- Je-li g vstupní brána proměnné x (tj. x je z množiny $\{x_1, \dots, x_n\}$), vytvoříme formuli $(\neg g \vee x) \wedge (g \vee \neg x)$. *Princip:* Aby tedy byla formule pravdivá, musí být g a x ohodnoceny stejně. Je totiž sémanticky ekvivalentní $g \leftrightarrow x$: formule $(\neg g \vee x)$ je totiž sémanticky ekvivalentní $(g \rightarrow x)$ a formule $(g \vee \neg x)$ je sémanticky ekvivalentní $(x \rightarrow g)$.
- Je-li g $\neg$ -brána se vstupem h_1 , vytvoříme formuli $(\neg g \vee \neg h_1) \wedge (g \vee h_1)$. (Která je sémanticky ekvivalentní $(g \leftrightarrow h_1)$.)
- Je-li g $\wedge$ -brána se vstupy h_1, h_2 , vytvoříme formuli $(\neg g \vee h_1) \wedge (\neg g \vee h_2) \wedge (\neg h_1 \vee \neg h_2 \vee g)$. (Která je sémanticky ekvivalentní $(h_1 \wedge h_2) \leftrightarrow g$.)
- Je-li g $\vee$ -brána se vstupy h_1, h_2 , vytvoříme formuli $(\neg h_1 \vee g) \wedge (\neg h_2 \vee g) \wedge (h_1 \vee h_2 \vee \neg g)$. (Která je sémanticky ekvivalentní $(h_1 \vee h_2) \leftrightarrow g$.)
- Je-li g výstupní bránou, pak navíc vytvoříme formuli g .

Principem celé konstrukce a základem formálního důkazu toho, že C a F jsou ekvivalentní vzhledem ke splnitelnosti, je fakt, že F kóduje správnost výpočtu pravdivosti bran v C .

Podrobněji. Nechť α je ohodnocení takové, že $\text{VARS}(\alpha) = \text{VARS}(C)$. Pokud $C[\alpha] = 1$, můžeme rozšířit α o ohodnocení proměnných $g_1, \dots, g_n$ tak, že pro výsledné ohodnocení α' máme $F[\alpha'] = 1$. Stačí proměnnou g ohodnotit na pravdivost $g[\alpha]$ v obvodu C .

Obráceně, pokud pro nějaké ohodnocení α máme $F[\alpha] = 1$, potom musí platit, že $\alpha(g)$ je stejně jako pravivost $g[\alpha]$ brány g v obvodu C (pro ohodnocení α). $\square$

2 Druhý seminář

Obsah. transformace CNF na 3CNF; transformace problému barvení na problém splnitelnosti; transformace sudoku na problém splnitelnosti.

~

def. Formule je v 3CNF, pokud je v CNF a každá klauzule obsahuje právě 3 literály. (Někdy definováno i tak, že klauzule obsahují nejvýše 3 literály, pro nás je to nepodstatný rozdíl.)

Transformace CNF na 3CNF. Transformujeme formuli F v CNF na formuli G v 3CNF. Předpokládáme

$$F = C_1 \wedge C_2 \wedge \dots \wedge C_m,$$

kde $C_1, C_2, \dots, C_m$ jsou klauzule. Formuli G vytvoříme jako konjunkci

$$G = G_1 \wedge G_2 \wedge \dots \wedge G_m,$$

kde G_i je formule v 3CNF získaná transformací klauzule C_i . Pokud nemá C_i více než 3 literály, dostaneme G_i dvojením nebo ztrojením některého z literálů, které C_i obsahuje. Jinak nechť

$$C_i = (l_1 \vee l_2 \vee \dots \vee l_k).$$

Potom

$$G_i = (l_1 \vee l_2 \vee z_1) \wedge (\neg z_1 \vee l_3 \vee z_2) \wedge \dots \wedge (\neg z_{k-3} \vee l_{k-1} \vee l_k),$$

kde $z_1, \dots, z_{k-3}$ jsou nové proměnné (budeme jim říkat z -proměnné a jejich literálům z -literály).

Otázka. Jsou F a G ekvivalentní vzhledem ke splnitelnosti?

Pokud je $C_i[\alpha] = 1$, musí existovat j tak, že $l_j[\alpha] = 1$. V G_i si vybereme klauzuli D obsahující l_j a z -literály v ní obsažené ohodnotíme na 0 (tj. jejich proměnné ohodnotíme tak, aby literály byly nepravdivé). Pokud klauzule D' sousedí s D , obsahuje pravdivý literál, který je negací nepravdivého z -literálu z D . Druhý z -literál v klauzuli D' tak ohodnotíme na 0 a v podobném duchu pokračujeme se sousedem D' . Nakonec budou pravdivé všechny klauzule: jedna díky l_j , zbylé díky pravdivosti z -literálů.

Pokud je $G[\alpha] = 1$ pro nějaké ohodnocení α , pak musí existovat literál l tak, že $l[\alpha] = 1$. Kdyby totiž pro všechny literály l bylo $l[\alpha] = 0$, musela by jít G splnit ohodnocením z -proměnných a to nejde. Musíme mít $\alpha(z_1) = 1$ (jinak bychom nesplnili první klauzuli formule G_i), $\alpha(z_2) = 1$ (jinak bychom nesplnili druhou klauzuli formule G_i) atd. Nakonec dostaneme $\alpha(z_{k-3}) = 1$ a poslední klauzule formule G_i tak není pravdivá.

Předchozí dva odstavce ukazují, že formule F a G vzhledem ke splnitelnosti ekvivalentní jsou. $\square$

def. Instancí problému barvení je neorientovaný graf s množinou vrcholů V a množinou hran H a přirozené číslo k . Cílem je zjistit, jestli existuje zobrazení $c : V \rightarrow \{1, 2, \dots, k\}$ takové, že pro každou $(v, w) \in H$ máme $c(v) \neq c(w)$.

Transformace problému barvení na splnitelnost formule v CNF. Na vstupu je graf $G = (V, H)$ a číslo k . Výsledkem je formule F v CNF. Předpokládáme $V = \{1, 2, \dots, n\}$.

Proměnné ve formuli F označíme x_{ij} pro $i = 1, 2, \dots, n$ a $j = 1, 2, \dots, k$. Hlavní myšlenka je, že ohodnocení α takové, že $F[\alpha] = 1$ bude odpovídat barvení: $\alpha(x_{ij}) = 1$ značí, že vrchol i je obarven barvou j .

Musíme zajistit, že formuli splní pouze ohodnocení α , které takto odpovídá korektnímu barvení. Toto vyjádříme pomocí tří podmínek. Platnost každé z nich vyjádříme formulí v CNF, formule F je potom konjunkcí těchto formulí.

1. *Každý vrchol je obarven alespoň jednou barvou.*

Pro vrchol i to znamená, že alespoň jedna proměnná z $\{x_{i1}, x_{i2}, \dots, x_{ik}\}$ musí být pravdivá. To zajistíme pomocí formule

$$\varphi_i = (x_{i1} \vee x_{i2} \vee \dots \vee x_{ik}).$$

(Pokud je pravdivá, alespoň jedna z proměnných je pravdivá).

2. *Každý vrchol je obarven nejvýše jednou barvou.*

Pro vrchol i to znamená, že nejvýše jedna proměnná z $\{x_{i1}, x_{i2}, \dots, x_{ik}\}$ je pravdivá. To zajistíme pomocí formule

$$\psi_i = \bigwedge_{1 \leq s < t \leq k} (\neg x_{is} \vee \neg x_{it})$$

(Pokud je pravdivá, nejvýše jedna z proměnných je pravdivá).

3. *Sousední vrcholy nejsou obarveny stejnou barvou*

Pro každou hranu $\{v, w\} \in H$ musí platit, že pro každou barvu j nejsou proměnné x_{vj} a x_{wj} obě pravdivé. To zajistíme pomocí formule

$$\kappa_{vw} = \bigwedge_{1 \leq j \leq k} (\neg x_{vj} \vee \neg x_{wj}).$$

Celkem tedy

$$F = \left(\bigwedge_{i=1}^n \varphi_i \wedge \psi_i \right) \wedge \bigwedge_{\{v,w\} \in H} \kappa_{vw}.$$

Z principu konstrukce plyne, že graf G lze obarvit k barvami, právě když je formule F splnitelná. Navíc každému ohodnocení α takové, že $F[\alpha] = 1$ odpovídá jedno barvení.

Formule JUST-ONE. Z výše uvedené konstrukce je užitečná formule

$$\text{JUST-ONE}(\{x_1, x_2, \dots, x_n\}) = (x_1 \vee x_2 \vee \dots \vee x_n) \wedge \left(\bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j) \right).$$

Tato formule je pravdivá jenom pro ohodnocení, které splní právě jednu proměnnou z množiny $\{x_1, x_2, \dots, x_n\}$.

Otázka. Lze podobně transformovat sudoku na splnitelnost formule v CNF?

Ano. Vstupem transformančního algoritmu je částečně doplněná sudoku tabulka o délce strany n . Výsledkem je formule F v CNF.

Zavedeme proměnné x_{ijk} pro $i, j, k = 1, 2, \dots, n$. Ohodnocení proměnné x_{ijk} na 1 značí, že na políčku na pozici i, j je zapsána hodnota k .

Dále pomocí konjunkce JUST-ONE formulí vyjádříme, že v každém sloupci, řádku a podtabulce je každá z hodnot $1, 2, \dots, n$ právě jednou. Ohodnocení, které splní tuto formuli, odpovídá správnému vyplnění tabulky hodnotami.

Počáteční hodnoty v tabulce doplníme pomocí dalších jednoprvkových klausulí. Je-li na pozici i, j hodnota k , dodáme k doposud vytvořené formulí klausuli (x_{ijk}) . $\square$

3 Třetí seminář

Obsah. význam problému splnitelnosti v teorii výpočetní složitosti; sat solvery; rezoluce;

~

Krátce jsme diskutovali složitostní třídy P, NP a roli problému splnitelnosti a (některých) transformací z minulých seminářů v otázce, jestli se tyto třídy rovnají.

~

V dalším se budeme na formuli v CNF dívat jako na množinu klausulí, přičemž klausule je množina literálů. V dalším narazíme na prázdnou množinu ve dvou situacích:

- jako na prázdnou formuli v CNF, tj. formuli neobsahující žádnou klausuli,
- jako na prázdnou klausuli, tj. klausuli neobsahující žádný literál.

Abychom tyto dvě situace odlišily, budeme v prvním případě používat značení $\emptyset$ a v druhém případě $\diamond$.

Otázka. Pro libovolné ohodnocení proměnných α máme $\emptyset[\alpha] = 1$ a $\diamond[\alpha] = 0$. Proč?

Disjunkce je pravdivá, když existuje alespoň jeden pravdivý disjunkt, přitom $\diamond$ nemůže žádný pravdivý disjunkt obsahovat pro žádné ohodnocení. Konjunkce je nepravdivá, pokud existuje alespoň jeden nepravdivý konjunkt. Ovšem $\emptyset$ neobsahuje nepravdivý konjunkt pro žádné ohodnocení, proto musí být pro všechna ohodnocení pravdivá. $\square$

def. Pro formuli F a ohodnocení α definujeme $F[\alpha]$ jako formuli, kterou získáme z F následovně:

- odstraníme všechny klauzule, které obsahují alespoň jeden literál, který α ohodnotí na 1.
- ze zbylých klausulí odstraníme všechny literály, které α ohodnotí na 0.

V případě, že $\text{vars}(\alpha) \supseteq \text{vars}(F)$, je značení zavedené předchozí definicí nejednoznačné. Podle dřívějších definic totiž máme $F[\alpha] \in \{0, 1\}$, ale podle nové definice je $F[\alpha] \in \{\emptyset, \{\diamond\}\}$. O kterou možnost se jedná, vždy poznáme z definice.

def. Nechť C_1, C_2 jsou klauzule a x je proměnná taková, že $x \in C_1$ a $\neg x \in C_2$. Rezolučním pravidlem aplikovaným na x, C_1, C_2 odvodíme klausuli

$$r(x, C_1, C_2) = (C_1 - \{x\}) \cup (C_2 - \{\neg x\}).$$

Této klausuli říkáme *resolventa*, klauzule C_1, C_2 jsou její *rodiče*. (Kromě označení *resolventy*, $r(x, C_1, C_2)$ také říká, že C_1 a C_2 obsahují různé literály proměnné x .)

def. Nechť F je množina klausulí. *Rezolučním odvozením* klauzule C z F je posloupnost klausulí

$$C_1, C_2, \dots, C_m$$

taková, že pro $i = 1, 2, \dots, m$ je buď $C_i \in F$, nebo $C_i = r(x, C_j, C_k)$ pro $j, k < i$; a navíc $C_m = C$. Délka odvození je rovna $m - |F|$.

def. Rezolučnímu odvození $\diamond$ z formule F říkáme *rezoluční zamítnutí* formule F .

Příklad. Mějme $F = \{C_1, C_2, C_3, C_4, C_5\}$, kde

$$\begin{aligned}C_1 &= \{x, \neg z\}, \\C_2 &= \{x, y, z\}, \\C_3 &= \{\neg x, \neg y, \neg z\}, \\C_4 &= \{\neg x, y\}, \\C_5 &= \{\neg y, z\}.\end{aligned}$$

Potom máme

$$\begin{aligned}C_6 &= r(z, C_1, C_2) = \{x, y\}, \\C_7 &= r(x, C_1, C_3) = \{\neg y, \neg z\}, \\C_8 &= r(x, C_6, C_4) = \{y\}, \\C_9 &= r(z, C_7, C_5) = \{\neg y\}, \\C_{10} &= r(y, C_8, C_9) = \diamond.\end{aligned}$$

Posloupnost $C_1, \dots, C_{10}$ je rezolučním zamítnutím F , má délku 5. □

Naším cílem je dokázat následující větu.

Věta 1 (O rezoluci). *Formule F je nesplnitelná, právě když existuje její rezoluční zamítnutí.*

Na promyšlení do příštího semináře jsme si nechali důkaz jedné z implikací ve větě o rezoluci, kterou zachycuje následující lemma.

Lemma 1 (O korektnosti rezoluce). *Pokud existuje rezoluční zamítnutí formule F , je formule F nesplnitelná.*

4 Čtvrtý seminář

Po promyšlení od minulého semináře dokazujeme větu o rezoluci.

Ekvivalenci dokážeme důkazem obou implikací.

Lemma 2 (O korektnosti rezoluce). *Pokud existuje rezoluční zamítnutí formule F , je formule F nespílitelná.*

Důkaz. Rezoluční pravidlo je korektní: pokud pro nějaké α máme $C_1[\alpha] = 1$ a $C_2[\alpha] = 1$, potom i $r(x, C_1, C_2)[\alpha] = 1$. Klausule C_1 a C_2 totiž musí každá obsahovat pravdivý literál, přičemž to nemůže být v obou případech literál proměnné x . Jeden z těchto pravdivých literálů tedy musí patřit do $r(x, C_1, C_2)$.

Nechť $C_1, \dots, C_t = \diamond$ je rezoluční zamítnutí formule F . Z předchozí odstavce plyne, že pokud pro nějaké ohodnocení α máme $F[\alpha] = 1$, máme i $C_i[\alpha] = 1$ pro všechna $i = 1, \dots, t$. Protože $\diamond$ je nespílitelná, takové α nemůže existovat a F je tedy nespílitelná. $\square$

def. Ohodnocení α , které ohodnocující právě literály $l_1, \dots, l_n$ vzájemně různých proměnných, budeme zapisovat $\{l_1 = \alpha(l_1); \dots; l_n = \alpha(l_n)\}$. Například $\alpha(x_1) = 1, \alpha(\neg x_2) = 0$ a $\alpha(x_3) = 0$ zapíšeme také $\{x_1 = 1; \neg x_2 = 0; x_3 = 0\}$.

Dokážeme následující pomocné tvrzení.

Lemma 3. *Nechť F je formule a l je literál, který se v ní vyskytuje. Nechť $C_1, \dots, C_t$ je rezoluční zamítnutí formule $F[[l = 1]]$. Potom existuje rezoluční odvození klausule C z formule F takové, že C je buď $\diamond$ nebo $\{\neg l\}$.*

Důkaz. Vytvoříme posloupnost $C'_1, \dots, C'_t$ následovně.

- Pokud $C_i \in F[[l = 1]]$, potom existuje $C \in F$ taková, že $C[[l = 1]] = C_i$. Vezmeme $C'_i = C$.
- Pokud $C_i = r(x, C_j, C_k)$, vezmeme $C'_i = r(x, C'_j, C'_k)$.

Indukcí snadno dokážeme, že pro každé i máme buď $C'_i = C_i$ nebo $C'_i = C_i \cup \{\neg l\}$ a že posloupnost $C'_1, \dots, C'_t$ je rezolučním odvozením C'_t z F . $\square$

Lemma 4 (O úplnosti vzhledem k zamítnutí). *Pokud je F nespílitelná, pak existuje její rezoluční zamítnutí.*

Důkaz. Dokážeme indukci přes $|\text{VAR}(F)|$.

- Pokud je $\text{VAR}(F)$ jednoprvková, pak musíme mít $F = \{\{x\}, \{\neg x\}\}$ pro nějakou proměnnou x . Rezolučním zamítnutím je posloupnost $\{x\}, \{\neg x\}, \diamond$.
- Předpokládejme, že tvrzení platí pro formule, které mají méně než $|\text{VAR}(F)|$ proměnných.

Z nespílitelnosti F plyne, že pro libovolnou proměnnou x jsou $F[x = 1]$ a $F[x = 0]$ taky nespílitelné. Tyto formule mají ovšem méně proměnných než F a můžeme na ně uplatnit indukční předpoklad. Na rezoluční zamítnutí formule $F[x = 1]$ a x uplatníme Lemma 3 a dostaneme odvození $\diamond$ nebo $\{\neg x\}$ z formule F . V prvním případě je důkaz hotov. V druhém případě aplikujeme Lemma 3 na $F[x = 0]$ a $\neg x$ a dostaneme odvození $\diamond$ nebo $\{x\}$. V prvním případě máme opět dokázáno. V druhém případě zřetězíme obě předchozí odvození za sebe a jedním použitím rezolučního pravidla odvodíme $r(x, \{\neg x\}, \{x\}) = \diamond$.

$\square$

Důkaz. (alternativní). Ukážeme, že pokud je F splnitelná, rezoluční zamítnutí neexistuje.

Nechť G je následující množina klausulí:

- $F \subseteq G$,
- G obsahuje právě všechny klausule, které lze odvodit z F .

Z druhé bodu plyne, $r(x, C_1, C_2) \in G$ pro každé vhodné klauzule $C_1, C_2 \in G$ a příslušnou proměnnou x (tj. je uzavřená na rezoluci). O množině G říkáme, že je rezolučním uzávěrem F .

Ukážeme, že pokud $\diamond \notin G$, pak je G splnitelná. Uděláme to tak, že najdeme ohodnocení α , které ji splní.

1. Začneme s α , které neohodnotí žádnou proměnnou.
2. Vybereme libovolné $x \in \text{VARS}(G)$. Pokud $\{x\} \in G$ nastavíme $\alpha(x) = 1$, jinak $\alpha(x) = 0$.
3. Nastavíme G na $G[x = \alpha(x)]$.
4. Opakujeme kroky 2,3 dokud $G \neq \emptyset$.

Nejdříve si všimneme, že $G[x = \alpha(x)]$ je uzavřená na rezoluci. Kdyby totiž existovali $C_1, C_2 \in G[x = \alpha(x)]$ a proměnná y tak, že $r(y, C_1, C_2) \notin G[x = \alpha(x)]$, nebyla by na rezoluci uzavřená ani G . Stejný rezoluční krok bychom mohli provést na klausule $C'_1, C'_2 \in G$ a proměnnou y , kde C'_i je buď C_i nebo $\neg C_i$, do které je přidán literál proměnné x (víme, že $x \neq y$). Resolventa by poté nepatřila do G , protože $r(y, C_1, C_2)$ nepatří do $G[x = \alpha(x)]$.

Pokud v kroku 2 máme $\{x\} \in G$, pak jistě $\{\neg x\} \notin G$, protože G je uzavřená na rezoluci a $\diamond \notin G$. (A obráceně, pokud $\{\neg x\} \in G$, pak $\{x\} \notin G$.) Tím pádem $\diamond \notin G[x = \alpha(x)]$. Algoritmus tedy skončí s prázdnou množinou. Během algoritmu jsme z G odstranili pouze pravdivé klausule (pro ohodnocení α), tedy všechny klauzule z F musí být také pravdivé. $\square$

Otázka. Lze předchozí důkaz transformovat na algoritmus pro splnitelnost?

Ano. Výpočet rezolučního uzávěru lze dělat následovně.

1. Nastavím $G \leftarrow F$.
2. Dokud jsem v minulé iteraci našel klausuli, která nebyla v G , opakuji:
3. pro všechny $C_1, C_2 \in G$ a proměnné x : pokud $C = r(x, C_1, C_2) \notin G$, přidám C do G .

Předchozí algoritmus jednou skončí, protože počet klausulí obsahujících pouze literály proměnných z $\text{VARS}(F)$ je konečný.

Pokud v průběhu výpočtu G narazíme na $\diamond$, formule není splnitelná. (Všechny klausule v jejím rezolučním zamítnutí jsou v G .) Jinak splnitelná je. Ohodnocení, které ji splní, lze nalézt algoritmem z důkazu. $\square$

5 Pátý seminář

def. Zvážíme následující varianty rezolučních odvození.

- *Pozitivní rezoluce.* Abychom mohli zařadit $r(x, C_1, C_2)$ do rezolučního odvození, alespoň jedna z klausulí C_1, C_2 obsahovat pouze pozitivní literály.
- *Unit rezoluce.* Abychom mohli zařadit $r(x, C_1, C_2)$ do rezolučního odvození musí být $|C_1| = 1$ nebo $|C_2| = 1$.

Otázka. Platí analogie věty o rezoluci i pro pozitivní rezoluci?

Ano. Tvrzení o korektnosti platí, protože pozitivní rezoluční odvození je speciálním případem obecného rezolučního odvození. Musíme ovšem ukázat úplnost vzhledem k zamítnutí, tj. ukázat, pokud je F je nesplnitelná, existuje její pozitivní rezoluční zamítnutí.

Tvrzení dokážeme indukcí přes $|\text{VARS}(F)|$.

- Je-li $|\text{VARS}(F)| = 0$, je $F = \{\Diamond\}$.
- Z nesplnitelnosti F plyne pro $x \in \text{VARS}(F)$ nesplnitelnost $F[\{x = 1\}]$ a nesplnitelnost $F[\{x = 0\}]$. Do pozitivního rezolučního zamítnutí formule $F[\{x = 0\}]$, které podle indukčního předpokladu existuje, vrátíme literál x jako v důkazu Lemma 3. Protože x je pozitivní literál, získaná posloupnost je odvozením pomocí pozitivní rezoluce. Pokud končí $\Diamond$, důkaz je u konce. Jinak musí posloupnost končit klauzulí $\{x\}$. Na konec posloupnosti postupně přidáme klauzule $r(x, \{x\}, C)$ pro všechny $C \in F$ obsahující $\neg x$. Tím jsou v posloupnosti všechny klauzule formule $F[\{x = 1\}]$. Na konec připojíme pozitivní rezoluční zamítnutí $F[\{x = 1\}]$ (ze kterého vynecháme klauzule obsažené v $F[\{x = 1\}]$), které existuje podle indukčního předpokladu. Získáme tak zamítnutí F pomocí pozitivní rezoluce.

□

Otázka. Platí analogie věty o rezoluci i pro unit rezoluci?

Neplatí. Unit rezoluce není úplná vzhledem k zamítnutí. Příkladem je libovolná kontradikce neobsahující klausuli s jedním literálem, třeba $\{\{x, y\}, \{\neg x, \neg y\}, \{\neg x, y\}\{x, \neg y\}\}$. □

def. Pro množinu klausulí F definujeme.

$$\begin{aligned} \text{WO}_x(F) &= \{C \in F \mid x \notin \text{VARS}(C)\} \\ \text{RES}_x(F) &= \{r(x, C_1, C_2) \mid C_1, C_2 \in F\} \\ \text{DP-STEP}_x(F) &= \text{WO}_x(F \cup \text{RES}_x(F)) \end{aligned}$$

Podle definice tedy $\text{DP-STEP}_x(F)$ dostaneme z F tak, že vybereme klauzule obsahující literál proměnné x , provedeme na nich (po dvojicích) rezoluční krok a resolventy přidáme do F . Poté z výsledné množiny odebereme všechny klauzule obsahující literál proměnné x . Máme tak $x \notin \text{VARS}(\text{DP-STEP}_x(F))$.

Pokud pro $\{x_1, x_2, \dots, x_k\} \subseteq \text{VARS}(F)$, odpovídá posloupnost

$$\begin{aligned} F_1 &\leftarrow \text{DP-STEP}_{x_1}(F) \\ F_2 &\leftarrow \text{DP-STEP}_{x_2}(F_1) \\ &\vdots \\ F_k &\leftarrow \text{DP-STEP}_{x_k}(F_k) \end{aligned}$$

(omezenému) rezolučnímu odvození. Na začátku vložíme do odvození všechny klausule F , poté postupně všechny klauzule z množiny $\text{RES}_{x_1}(F)$, tím jsou vlastně v odvození všechny klauzule z F_1 . Poté totéž opakujeme pro F_2 a dostaneme klauzule z F_3 atd. Přitom se v suffixu rezolučního odvození od konce odvození F_i nevyskytuje rezoluční krok podle proměnné x_i . Následující věta ukazuje, že výše znázorněné opakování DP-STEP je ve skutečnosti algoritmem pro rozhodnutelnost (ve kterém nemusíme konstruovat revoluční uzávěr).

Věta 2 (DP rezoluce je algoritmus pro test splnitelnosti). *Nechť F je formule a $x \in \text{VARS}(F)$. Potom je F splnitelná, právě když je $\text{DP-STEP}_x(F)$ splnitelná.*

Důkaz. Připomeneme základní fakt, že pro libovolnou formuli G , klauzuli $C \in G$ a ohodnocení α máme, že $G[\alpha] = 1$ implikuje $(G - \{C\})[\alpha] = 1$.

To, že splnitelnost $\text{DP-STEP}_x(F)$ plyne ze splnitelnosti F , je důsledkem korektnosti rezolučního kroku a předchozího faktu.

Předpokládejme, že $\text{DP-STEP}_x(F)$ je pravdivá pro ohodnocení α , které neohodnocuje proměnnou x (tj. $x \notin \text{VARS}(\alpha)$). Současně můžeme předpokládat, že F neobsahuje tautologické klauzule (ty nemohou ohrozit pravdivost F).

Klausule formule F neobsahující x , jsou pro α pravdivé, protože $\text{WO}_x(F) \subseteq \text{DP-STEP}_x(F)$ (viz fakt na začátku důkazu).

Nechť $D \in F$ je klausule, kterou α nesplní (tj. $D[\alpha] \neq 1$). Kdyby D neexistovala, je důkaz u konce. Z předchozího odstavce plyne, že D musí obsahovat literál l proměnné x . Klauzule obsahující literál l (jedna z nich je D) splníme nastavením $\alpha(l) = 1$.

Zbývají klauzule obsahující $\neg l$. Za tímto účelem zvažíme množiny

$$\begin{aligned} F' &= \{C \in F \mid \neg l \in C\}, \\ F'' &= \{r(x, D, C) \mid C \in F'\}. \end{aligned}$$

Protože $F'' \subseteq \text{RES}_x(F)$ a $\text{RES}_x(F)[\alpha] = 1$, máme $F''[\alpha] = 1$. Protože $(D - \{l\})[\alpha] = 0$, musí být každá $r(x, D, C)$ pro $C \in F'$ splněna literálem z C , tedy $C[\alpha] = 1$. Odtud plyne $F'[\alpha] = 1$. $\square$

Speciální případy s algoritmy v polynomickém čase

Pro formule v 2CNF je algoritmus z konce minulého semináře algoritmem pracujícím v polynomickém čase. Pokud mají klauzule C_1, C_2 nejvýše 2 literály, má jejich rezolventa také nejvýše dva literály. Rezoluční uzávěr formule s n proměnnými tak má $O(n^2)$ klauzulí.

def. Formule F je *Hornova*, pokud každá její klauzule obsahuje nejvýše jeden pozitivní literál.

Klausule v Hornově formuli interpretujeme jako implikace.

Interpretace (např. logické programování)	
$(x \vee \neg y_1 \vee \dots \vee \neg y_k)$	pravidlo $(y_1 \wedge \dots \wedge y_k) \rightarrow x$
(x)	pozitivní fakt $(1 \rightarrow x)$
$(\neg x)$	negativní fakt $(x \rightarrow 0)$

6 Šestý seminář

Algoritmus. Vstup je Hornova formule F .

1. Dokud existuje $\{x\} \in F$, nastav $F \leftarrow F[\{x = 1\}]$.
2. Pokud $\diamond \in F$, je F nesplnitelná.
3. F je splnitelná.

Otázka. Je algoritmus korektní? Jakou má složitost?

Složitost algoritmu je polynomická: cyklus v kroku 1 proběhne maximálně $|\text{VARS}(F)|$ -krát, každá iterace projde formuli F jednou. Kroky 2 a 3 lze provést jedním průchodem formulí F .

Korektnost algoritmu: Pokud je na řádce 1 formule F splnitelná, je splnitelná i $F \leftarrow F[x = 1]$. Pokud na řádce 2 najdeme $\diamond$, mohlo se to podle definice pravivosti F stát jenom tak, že jsme v nějaké klauzuli ohodnotily všechny literály na 0 a tím pádem je nepravdivá. Na řádce 3 obsahuje každá klausule aspoň jeden negativní literál, lze ji tedy splnit ohodnocením všech proměnných na 0.

Alternativně se lze na algoritmus dívat následovně: Krok 1 opakovaně aplikuje unit rezoluci (kde navíc jedna unit klauzule obsahuje pozitivní literál). Pokud se povede odvodit $\diamond$, je formule nesplnitelná (díky korektnosti resolučního kroku). Pokud nejde aplikovat unit rezoluci, obsahuje každá klauzule alespoň jeden negativní literál. $\square$

Jednoduchý backtracking

BACKTRACK(F)

1. Pokud $\diamond \in F$, vrať 0
2. Pokud $F = \emptyset$, vrať 1
3. Vyber $x \in \text{VARS}(F)$.
4. Pokud BACKTRACK($F[x = 1]$) = 1, vrať 1.
5. Vrať BACKTRACK($F[x = 0]$).

Věta 3. Nechť F je kontradikce a k je minimální možný počet rekurzivních volání procedury BACKTRACK (pro všechna možná konkrétní řešení řádku 3) pro formuli F . Potom existuje rezoluční zamítnutí formule F s nejvýše k klauzulemi.

Důkaz. Nechť T je strom rekurzivních zavolání BACKTRACK(F). Každé hraně v tomto stromu přiřadíme ohodnocení nějaké proměnné podle řádků 4 a 5. Všechny vrcholy v T mimo listů mají dva potomky.

Pro vrchol s je $p(s)$ ohodnocení proměnných, které je dáno ohodnoceními na cestě z kořene do s . (Platí tak například, že $|\text{VARS}(p(s))|$ se rovná hloubce s .)

Každému vrcholu s přiřadíme klausuli C_s tak, že $C_s[p(s)] = 0$. Kořeni tak musíme přiřadit klausuli $\diamond$, protože to je jediná klausule nepravdivá pro prázdné ohodnocení. Klausule najdeme strukturální indukcí.

Pokud je s listem, musí existovat klauzule $C \in F$ taková, že $C[p(s)] = \diamond$ (řádek 1 algoritmu). Vezmeme $C_s = C$.

Pokud s není list, má potomky t, v (ke kterým už díky indukčnímu předpokladu máme C_t a C_v).
Nechť x je proměnná, která je ohodnocena na hranách z s do t, v .

- Pokud $x \in \text{VARS}(C_t) \cap \text{VARS}(C_v)$, potom $x \in C_t$ a $\neg x \in C_v$, protože podle indukčního předpokladu $C_t[p(t)] = 0$ a $C_v[p(v)] = 0$. Vezmeme $C_s = r(x, C_t, C_v)$. Máme $C_s[p(s)] = 0$, protože C_t a C_v neobsahují žádný pravdivý literál a $p(s)$ se shoduje s $p(t)$ a $p(v)$ v ohodnocení všech proměnných mimo x .
- Pokud $x \notin \text{VARS}(C_t) \cap \text{VARS}(C_v)$, je C_s ta z klauzulí C_t, C_v , která neobsahuje literál proměnné x . V tomto případě je triviálně $C_s[p(s)] = 0$

Rezoluční zamítnutí fle F dostaneme tak, že klauzule odpovídající vrcholům seřadíme v opačném pořadí, než ve kterém bychom prošli vrcholy stromu T při průchodu do hloubky. $\square$

Existuje rodina formulí, pro které má rezoluční zamítnutí exponenciální délku (vzhledem k počtu proměnných). Z toho plyne, že algoritmus BACKTRACK (a další algoritmy od něj odvozené) mají exponenciální složitost. Důkaz existence takové rodiny si řekneme později.

DPLL a varianty

DPLL(F)

1. Pokud $\diamond \in F$, vrať 0
2. Pokud $F = \emptyset$, vrať 1
3. Dokud $\{l\} \in F$ nastav $F \leftarrow F[l = 1]$.
4. S nějakou strategií vybere literál l proměnné z $\text{VARS}(F)$.
5. Pokud $\text{DPLL}(F[l = 1]) = 1$, vrať 1.
6. Vrať $\text{DPLL}(F[l = 0])$.

Lze chápat jako BACKTRACK s přidanou *unit propagací* a strategií výběru proměnné a jejího ohodnocení. Tady existuje mnoho heuristik, uvedme si některé z nich. Naše příklady jsou založeny na

$$f_k(u) = |\{C \mid u \in C, |C| = k\}|,$$

$$f(u) = \sum_{k \geq 1} f_k(u).$$

Heuristika	Princip
DLIS	Vybereme literál l , který maximalizuje $f(l)$.
DLCS	Vybereme proměnnou x , která maximalizuje $f(x) + f(\neg x)$. Pokud $f(x) + f(\neg x)$, vybereme literál x , jinak literál $\neg x$.
MOM	Nechť k je velikost nejmenší klauzule. Vybereme proměnnou x , která maximalizuje $(f_k(x) + f_k(\neg x)) \cdot p + f_k(x)f_k(\neg x)$, kde $p > \frac{f_k(x)f_k(\neg x)}{f_k(x) + f_k(\neg x)}$.

1. Pokud $\diamond \in F$, vrať 0
2. Pokud $F = \emptyset$, vrať 1
3. Vyber nejkratší klausuli $C = \{l_1, l_2, \dots, l_m\}$.
4. pro $i = 1, \dots, m$:
pokud je $\text{MONIEN-SPECKENMEYER-1}(F[\{l_1 = 0, l_2 = 0, \dots, l_i = 1\}]) = 1$, vrať 1
5. vrať 0.

Algoritmus je speciální verzí DPLL, kde vybíráme ohodnocení proměnné z krátkých klauzulí. Algoritmus automaticky dělá unit propagaci.

Složitost. Předpokládejme, že algoritmus je spuštěn na formuli v kCNF. Složitost určíme jako počet rekurzivních zavolání. Jako $T(n)$ označíme složitost algoritmu pro formuli s n proměnnými. Z algoritmu vidíme

$$T(n) \leq T(n-1) + T(n-2) + \dots + T(n-m).$$

Pokud $m \leq k$ a předpokládáme $T(n) = a^n$ pro nějaké $a > 1$, potom pro $m = k$ (po zkrácení obou stran rovnosti a^{n-k} dostaneme

$$a^k = a^{k-1} + \dots + a + 1.$$

S použitím vzorečku $a^{k-1} + \dots + a + 1 = \frac{1-a^k}{1-a}$ dostaneme po dosazení $a^{k+1} + 1 = 2a^k$. Řešení této rovnice pro různá k jsou v následující tabulce. (*Zajímavý údaj je také $b = \log_2 a$. Platí totiž $a^n = 2^{bn}$. Vidíme tak podíl proměnných, které jsou ohodnoceny, pokud porovnáme s algoritmem typu zkusím všechna ohodnocení.*)

k	3	4	5
a	1.839	1.920	1.966
b	0.879	0.94	0.97

def. Ohodnocení α je *autark* pro F , pokud pro každé $C \in F$ máme, že $\text{vars}(C) \cap \text{vars}(\alpha) \neq \emptyset$ implikuje $C[\alpha] = 1$.

Pro ohodnocení α , které je autark pro F , je $F[\alpha] \subseteq F$. Tedy, pokud je tedy F splnitelná, je splnitelná i $F[\alpha]$. Připomeňme, že obrácený směr platí obecně (α nemusí být autark), je-li splnitelná $F[\alpha]$, pak je splnitelná i F .

MONIEN-SPECKENMEYER-2(F)

1. Pokud $\diamond \in F$, vrať 0
2. Pokud $F = \emptyset$, vrať 1
3. Vyber nejkratší klausuli $C = \{l_1, l_2, \dots, l_m\}$.
4. pro $i = 1, \dots, m$:
 $\alpha \leftarrow \{l_1 = 0, l_2 = 0, \dots, l_i = 1\}$
pokud je α autark, vrať MONIEN-SPECKENMEYER-2($F[\alpha]$)
5. pro $i = 1, \dots, m$:
pokud je $\text{MONIEN-SPECKENMEYER-2}(F[\{l_1 = 0, l_2 = 0, \dots, l_i = 1\}]) = 1$, vrať 1
6. vrať 0.

Autark ohodnocení je použito analogicky unit propagaci.

Složitost. Předpokládejme, že algoritmus je spuštěn na formuli v kCNF. Musíme vzít v úvahu to, jestli jsme našli autark ohodnocení. Pokud jsme našli autark ohodnocení, potom $T(n) = T(n-1)$. Situaci, kdy jsme autark ohodnocení nenašli, obsahuje $F[\{l_1 = 0, l_2 = 0, \dots, l_i = 1\}]$ vždycky formuli s nejvýše $k-1$ literály (α není autark, takže v F existuje klauzule, jíž α ohodnotilo literál, ale klauzuli nesplnilo). Máme tedy

$$T(n) = \max\{T(n-1), T'(n-1) + \dots + T'(n-k)\},$$

kde pomocí T' značíme funkci složitosti v situaci, kdy víme, že formule obsahuje klauzuli s nejvýše $k-1$ literály.

Podobné úvahy nyní provedem pro T' a dostaneme

$$T'(n) = \max\{T(n-1), T'(n-1) + \dots + T'(n-k+1)\}.$$

(Máme méně rekurzivních volání, klauzule C má nejvýše $k-1$ literálů. V situaci, kdy najdeme autark ohodnocení, ovšem existenci klauzule s méně než k literály nemůžeme předpokládat.)

Indukcí přes n bychom dokázali, že

$$T(n-1) \leq T'(n-1) + \dots + T'(n-k+1),$$

a vezmeme tak rekurenci

$$\begin{aligned} T(n) &\leq T'(n-1) + \dots + T'(n-k+1), \\ T'(n) &\leq T'(n-1) + \dots + T'(n-k+1). \end{aligned}$$

(Tj. v nejhorším případě nenajdeme autark ohodnocení, víme ale, že v rekurzivních zavoláních je klauzule, která má nejvýše $k-1$ literálů. Vidíme taky, že $T(n) \in O(T'(n))$, protože k je konstanta.)

Předpokládáme $T'(n) = a^n$ a z druhé nerovnosti dostaneme

$$a^{k-1} = a^{k-2} + \dots + a + 1$$

a použitím stejného postupu jako předtím vypočítáme

$$a^k + 1 = 2a^{k-1}.$$

Dostaneme tak tabulku

k	3	4	5
a	1.618	1.839	1.928
b	0.694	0.879	0.947

7 Sedmý seminář

Základy implementace DPLL v solverech

Tento seminář je založen na zjednodušení solveru MiniSat (*odkaz je na webu semináře*)

Následující pojmy se týkají jedné větve výpočtu v rekurzivním stromu procedury DPLL. Proměnná je *rozhodovací*, pokud není ohodnocena unit-propagací. *Level proměnné* x je definován jako počet rozhodovacích proměnných, které byly ohodnoceny před x (jenom těch ze stejné větve výpočtu).

Reprezentace entit.

- Proměnné: čísla 1, 2, 3, 4
- Literály proměnné k jsou $2k$ (pozitivní) a $2k + 1$ (negativní)
- Proměnná literálu l je $\text{var}(l)$: $\text{floor}(l/2)$
- Negace literálu l je $\text{neg}(l)$: $l \text{ xor } 1$
- Test positivity literálu l je $\text{sgn}(l)$: $(l \ \& \ 1) = 0$

Hodnoty proměnných (a literálů) v solveru jsou TRUE, FALSE, NONE (bez ohodnocení).

Klauzuli reprezentujeme jako pole literálů. První dva prvky v poli jsou tzv. *sledované* (anglicky watched) literály. Sledovaný literál je literál, o kterém hlídáme, že není FALSE (a klauzule jej obsahující také není FALSE).

Když tedy literál l ohodnotíme na TRUE, musíme ve všech klauzulích, ve kterých je $\text{neg}(l)$ sledováním literálem, situaci nějak vyřešit. Mohou nastat následující případy:

- Druhý ze sledovaných literálů v klauzuli je ohodnocen NONE, označme jej k . Literál $\text{neg}(l)$ se pokusíme nahradit nesledovaným literálem z klauzule, který není FALSE. Pokud to jde, s úpravou končíme. Pokud se to nepovede, jsme nuceni ohodnotit literál k unit propagací na TRUE.
- Druhý ze sledovaných literálů je FALSE. To se stane pouze v případě, že jsme l ohodnotily pomocí unit propagace. Klauzule je tedy v současném ohodnocení nepravdivá. O této situaci říkáme, že došlo ke *konfliktu*.
- Druhý ze sledovaných literálů je TRUE. Neděláme žádné změny, protože nemůže dojít ke konfliktu.

Abychom uměli efektivně najít všechny klauzule, ve kterých je nějaký literál sledován, udržujeme si ke každému literálu seznam všech klauzulí, ve kterých je sledován. Při ohodnocení tohoto literálu na FALSE pak stačí tento seznam projít. (V implementaci níže si u literálu p pamatujeme klauzule, ve kterých je sledována negace p .)

Algoritmus není realizován rekurzivně, ale iterativně. Jeho hlavní smyčka vypadá typicky následovně.

```
1 search()
2 -----
3 while
4     conflict <- propagate()
5     if conflict
6         ok <- backtrack()
7         if (not ok) then return UNSAT
8     else
9         if (len(trail) = vars) then return SAT
```

```

10     else
11         pick a literal p
12         assume p

```

Proměnné ohodnocujeme tak, že udržeme frontu literálů, které chceme ohodnotit na TRUE. Procedura `propagate` si literály z této fronty postupně bere, a promítne toto ohodnocení do datových struktur, zejména do sledovaných literálů. Toto může vést ke konfliktu (viz diskuze o sledovaných literálech), v takovém případě procedura vrací `false`. Pokud to nevede ke konfliktu, vrací procedura `true`. Volání procedury může vést k ohodnocení více proměnných, je to díky unit propagaci (viz diskuze o sledovaných literálech).

Pokud `propagate` vrací `false`, provádíme backtracking. To je proces, ve kterém zapomeneme ohodnocení některých proměnných. To odpovídá návratu z rekurzivního zavolání v algoritmu DPLL. Pokud už backtracking nejde provést (jsme v kořeni stromu rekurze), vrací `backtrack` hodnotu `false` a formule není splnitelná a algoritmus vrací `UNSAT`.

Pokud `propagate` vrací `true`, může algoritmus pokračovat k ohodnocení dalších proměnných. V případě, že už ohodnotil všechny, je formule splnitelná a algoritmus vrací `SAT`. Jinak pokračuje výběrem další rozhodovací proměnné a jeho literálu.

Před kódem dalších procedur uvedeme globální data, která procedury používají.

- `vars` je počet proměnných
- `clauses` jsou klauzule. Jedna klauzule je přitom pole literálů (viz diskuze výše). Při implementaci lze uložení klauzulí řešit různě.
- `watches` je pole. Prvek je `watches[l]` je kolekce klauzulí, ve kterých je `neg(l)` sledovaným literálem.
- `prop-q` je fronta literálů, které jsme ohodnotili na TRUE a které čekají, až jejich ohodnocením pomocí `propagate` promítne do klauzulí (zejména sledovaných literálů, viz diskuze výše).
- `assigns` je pole ohodnocení proměnných. Hodnota `assigns[i]` je ohodnocení proměnné `i`, je jedna z hodnot `TRUE`, `FALSE`, `NONE`. Pole má velikost `vars + 1`. (Prvky jsou inicializovány na `NONE`.)
- `levels` je pole levelů jednotlivých proměnných. Tj. prvek `levels[i]` je level proměnné `i`. Pole má velikost `vars + 1`.
- `tries` je pole velikosti `vars + 1`. Hodnota `vars[i]` je počet ohodnocení, které jsme pro proměnnou `i` v aktuální větvi výpočtu vyzkoušeli (tento počet je 0, 1, nebo 2. Používá se primárně pro rozhodovací proměnné.
- `trail` je zásobník obsahující literály ohodnocené na TRUE. Zásobník je implementován polem. Velikost zásobníku je tak počet ohodnocených proměnných.
- `trail-lim` je zásobník indexů do `trail`. Na těchto indexech jsou v `trail` uloženy literály rozhodovacích proměnných. Velikost zásobníku je tak aktuální level.

Argumentem procedury `assume` je literál, který ohodnocujeme na TRUE. Tato procedura je volána pouze v případě, že se literál není ohodnocen unit propagací, tj. je to literál rozhodovací proměnné.

```

1  assume(p)
2  -----
3  push(trail_lim, length(trail))
4  return enqueue(p)

```

Procedura `enqueue` zařazuje literál `p` do fronty `prop-q` a vrací jestli se to povedlo. Na začátku nejdříve zkontroluje, jestli je daná proměnná již ohodnocena a jestli je případné ohodnocení v konfliktu a argumentem procedury.

```

1 enqueue(p)
2 -----
3 if (value(p) == TRUE) then return true
4 if (value(p) == FALSE) then return false
5
6 v <- var(p)
7 assigns[v] <- sgn(p) ? TRUE : FALSE
8 level[v] <- length(trail-lim)
9 tries[v] += 1
10 insert p into prop-q
11 push p to trail
12 return true

```

Procedura `propagate` bere literály z fronty `prop-q` (které jsme ohodnotily na `TRUE`) a jejich ohodnocení promítá do klauzulí a do sledovaných literálů. Pokud při ohodnocení dojde ke konfliktu (viz diskuze výše), vrací `false`, jinak vrací `true`. Pro literál `p` (získaný z `prop-q`) promítne jeho ohodnocení do všech klauzulí v `watches[p]`. (Přitom `watches[p]` vyprázdní). Když dojde v nějaké klauzuli ke konfliktu, vrátí nezpracované klauzule zpět do `watches[p]`, vyprázdní `prop-q` a vrátí `false`.

```

1 propagate()
2 -----
3 while prop_q is not empty
4   p <- dequeue(prop_q)
5   tmp <- watches[p]
6   remove all from watches[p]
7   for i in 0..

```

Procedura `propagate-clause` promítne ohodnocení literálu `p` na `TRUE` do klauzule `cl`. Je volána pouze v situaci, že `neg(p)` je v klauzuli `cl` sledovaným literálem. Procedura nejdříve zajistí, že `neg(p)` je na indexu 1 a pokusí jej nahradit nesledovaným literálem, který není `FALSE`. Pokud se to povede vrátí procedura `true`. Jinak vrátí `cl` do `watches[p]` (odkud jej procedura `propagate` odebrala) a zavolá `enqueue` na druhý sledovaný literál, to je `cl[0]`, a vrátí jeho výsledek. Pokud je hodnota literálu `cl[0]` rovna `FALSE`, procedura `enqueue` vrátí `false` a správně detekujeme konflikt. Jinak je `cl[0]` přidán do `prop-q`. Jedná se tak o unit propagaci.

```

1 propagate-clause(cl, p)
2 -----
3 if (cl[1] != neg(p)) then swap(cl[0], cl[1])
4 if (value(cl[0]) = TRUE) then return true
5
6 for i in 2..length(cl)
7   if (value(cl[i]) != FALSE)
8     swap(cl[i], cl[1])
9     insert cl into watches[neg(cl[1])]
10   return true
11 insert cl into watches[p]
12 return enqueue(cl[0])

```

Procedura `undo-one` zruší ohodnocení posledního ohodnoceného literálu. Toto se promítne do všech potřebných globálních dat. V klauzulích a sledovaných literálech není potřeba měnit nic. Jestli je `p` literál rozhodovací proměnné lze zjistit pomocí `trail-lim`.

```

1  undo-one()
2  -----
3  p <- pop(trail)
4  v <- var(p)
5  if p is decision variable then pop(trail_lim)
6  assigns[v] <- NONE
7  level[v] <- -1
8  tries[v] <- 0

```

Procedura `backtrack` volá `undo-one` dokud nenarazí na literál rozhodovací proměnné, který nebyl ohodnocen dvakrát. Jeho proměnnou ohodnotí na nepoužitou hodnotu a vrátí `true`. Pokud vyprázdní `trail-lim` vrací `false`. Nepodařilo se úspěšně ohodnotit proměnnou odpovídající kořeni stromu rekurze algoritmu DPLL.

```

1  backtrack()
2  -----
3  while
4    if (length(trail-lim) = 0) then return false
5    while top(trail) is not decision variable do undo-one()
6    p <- top(trail)
7    if (tries[var(p)] < 2) and enqueue(neg(p)) then return true
8    else undo-one()

```

Zbývá okomentovat princip výběru rozhodovacích proměnných a jejich literálů (ř. 11 procedury `search`). V principu lze použít libovolnou heuristiku uvedlou u DPLL algoritmu, je ovšem nutno zajistit její efektivní implementaci.

Solver MiniSat používá heuristiku založenou na aktivitách proměnných. Každá proměnná má přiřazenu aktivitu, to je kladné číslo. Na začátku je tato aktivita rovna 0. Po každém konfliktu se proměnným, které se konfliktu zúčastnili (viz analýza konfliktu níže), zvýší aktivita o inkrement x . Další částí je, že ostatním proměnným by se měla aktivita exponenciální rychlostí snižovat. Toto by ovšem bylo výpočetně náročné, proto je nahrazeno postupným zvyšování inkrementu, podle pravidla

$$x \leftarrow x \frac{1}{d},$$

kde $0 < d < 1$ (typicky mezi 0.8 a 0.99, v MiniSat $d = 0.95$). Jednou za čas, když hrozí overflow, vynásobíme všechny aktivity a inkrement malým číslem. Jako další proměnná k ohodnocení je vybrána ta s největší aktivitou. (Toto je efektivně implementováno pomocí prioritní fronty, např. binary heap.) Ideou heuristiky je pokusit se ohodnotit proměnné způsobující konflikty co nejdříve (představují obtížnou část formule).

8 Osmý seminář

Hlavní myšlenky CDCL solveru

Rozšíření DPLL solveru o *učení se nových klauzulí* a *nechronologický backtracking*.

Konflikt je, když je nějaká klauzule vyprázdněna (enqueue a tedy propagate vrátí false). Tuto klauzuli označujeme jako konfliktní.

Nechť je konfliktní klauzule rovna $\{l_1, l_2, \dots, l_k\}$. Konflikt je způsoben pravdivostí literálů $\neg l_1, \neg l_2, \dots, \neg l_k$. Tyto zpracujeme následovně, vytváříme při tom naučenou klauzuli C , která je na začátku prázdná.

- je-li proměnná literálu $\neg l_i$ rozhodovací, přidáme $\neg l_i$ do učené klauzule.
- je-li $\neg l_i$ splněn unit propagací, potom existovala klauzule $\{\neg l_i, l'_1, l'_2, \dots\}$, která byla transformována na $\{\neg l_i\}$, což vedlo unit propagaci. Do vytvářené klauzule tak uvážíme přidání literálů $\neg l'_1, \neg l'_2, \dots$, tyto ovšem musí opět projít oběma kroky. (Toto se dá dělat jemněji.)

Klauzuli C , která obsahuje pouze literály rozhodovacích proměnných, přidáme ke vstupní formuli. Tím v budoucnu předejdeme konfliktu způsobenému stejnou klauzulí jako nyní.

Vybereme proměnnou, která má literál v C a má tam maximální level. Provedeme backtracking až do místa, kde tuto proměnnou ohodnocujeme.

Naučené klauzule se mohou začít akumulovat, je vhodné je začít zapomínat, aby formule nebyla příliš dlouhá. K tomu lze použít řadu heuristik, blíže to ale diskutovat nebudeme.

Paturi-Pudlák-Zane algoritmus

PPZ(F)

1. α nastavíme na prázdné ohodnocení (tj. $\text{VARS}(\alpha)$).
2. Vybereme náhodnou permutaci π množiny $\{1, 2, \dots, n\}$.
3. pro i rovno postupně $1 \dots n$
 - $j \leftarrow \pi(i)$
 - Pokud $\{x_j\} \in F$, pak nastav $\alpha(x_j) \leftarrow 1$.
 - Pokud $\{\neg x_j\} \in F$, pak nastav $\alpha(x_j) \leftarrow 0$.
 - Jinak náhodně vybereme $a \in \{0, 1\}$ a nastavíme $\alpha(x_j) \leftarrow a$.
 - $F \leftarrow F[\alpha]$.
4. Vrať α .

Jedná se v podstatě o pravděpodobnostní verzi algoritmu DPLL, kde pořadí proměnných a jejich ohodnocení (to mimo unit propagaci), vybíráme náhodně.

Algoritmus může vrátit ohodnocení α tak, že $F[\alpha] = 0$ i v případě, že F je splnitelná. V opačném směru chybu neudělá: je-li F kontradikce, pak algoritmus nikdy nevrátí ohodnocení, které F splní (ono ani neexistuje).

Označíme p pravděpodobnost toho, že algoritmus neudělá chybu (tj. vrátí ohodnocení, pro které je F pravdivá). Pravděpodobnost toho, že algoritmus t krát zopakujeme a při každém opakování

algoritmus udělá chybu, je rovna $(1 - p)^t$. Protože platí $(1 - p)^t \leq e^{-tp}$, abychom pro konstantu c dostali pravděpodobnost chyby pod e^{-c} , musíme provést $\frac{c}{p}$ opakování.

O algoritmu platí následující věta (důkaz vynecháme).

Věta 4. *Pravděpodobnost toho, že PPZ vrátí pro splnitelnou F ohodnocení α , pro které je pravdivá, není menší než $2^{-n(1-1/k)}$, kde k maximální velikost klauzule a n je počet proměnných v F .*

Podle úvahy nad větou, chceme-li dosáhnout konstantní chyby (např. $\frac{1}{e^d}$) pro konstantu d , stačí algoritmus zopakovat $d \cdot 2^{n(1-1/k)}$ krát, celková složitost je tak $O(2^{n(1-1/k)} \cdot f(n))$, kde $f(n)$ je složitost jednoho běhu PPZ.

k	3	4	5
$2^{1-1/k}$	1.587	1.682	1.741
$1 - 1/k$	0.667	0.75	0.8

Zrychlení lze dosáhnout dalším vylepšením: k F doplníme všechny resolventy klauzulí z F , které mají méně než s literálů, kde $s = o(\frac{n}{\log n})$ a až poté spustíme algoritmus. Pro 3CNF dostaneme algoritmus se složitostí $O(1.308^n)$.

Lower bound délky resolučního zamítnutí

Lemma 5. *Nechť R je rezoluční zamítnutí formule F a l je literál vyskytující se v F . Potom existuje rezoluční zamítnutí R' formule $F[\{l = 1\}]$, jehož klauzule neobsahují $l, \neg l$ a $|R'| \leq |R| - |F_l|$, kde F_l je množina klauzulí formule F , které obsahují l .*

Důkaz. Označíme $R = C_1, C_2, \dots, C_t = \diamond$. Začneme s prázdným R' , a poté pro $j = 1, 2, \dots$ se na základě C_i rozhodneme, zda-li z ní (úpravou) vytvoříme C'_i a zařadíme ji na konec R' . Na konci tohoto procesu je R' rezoluční zamítnutí s požadovanými vlastnostmi.

Pravidla úprav.

1. Pokud $C_i \in F$ a $l, \neg l \notin C_i$, vezmeme $C'_i = C_i$.
2. Pokud $C_i \in F$ a $\neg l \in C_i$, pak $C'_i = C_i - \{\neg l\}$.
3. Pokud $l \in C_i$, je C'_i vynechána.
4. Pokud $C_i = r(y, C_j, C_k)$ a C'_j, C'_k jsou v R' a $y \in C'_j$, $\neg y \in C'_k$, potom $C'_i = r(y, C'_j, C'_k)$.
5. Pokud $C_i = r(y, C_j, C_k)$ a C'_j, C'_k jsou v R' , ale $y \notin \text{vars}(C_j)$, potom $C'_i = C'_j$.
6. Pokud $C_i = r(y, C_j, C_k)$, ale C'_j není v R' , protože byla vynechána v kroku 3. Máme tak $l \in C_j$ a $l \notin C_i$ (nebyl krok 3). Musíme tak mít $y \in \text{vars}(l)$. Proto máme $\neg l \in C_k$ a $\neg l \notin C'_k$, protože $\neg l$ vypadlo z C_k pomocí kroku 2 (pokud $C_k \in F$), nebo pomocí kroku 2 aplikovaného na některého z resolučních předků C_k . Vezmeme $C'_i = C'_k$.

Pokud C'_i není vynechána, platí $C'_i \subseteq C_i$. Protože $C'_i \subseteq C_t = \diamond$, je R' rezoluční zamítnutí.

Po provedení úprav vynecháme opakující se klauzule. Vynecháme minimálně klauzule v bodě 3, tedy přesně F_l . $\square$

Dirichletův princip. Nechť $n \in \mathbb{N}$ a $n > 1$. Označíme $[n] = \{1, 2, \dots, n\}$. Neexistuje bijekce mezi $[n]$ a $[n - 1]$. (Někdy označováno jako *pigeonhole principle*, dále mluvíme o n holubech a $n - 1$ dírách.)

Každé přiřazení n holubů do $n - 1$ děr zachytíme pomocí matice $(x_{ij})_{n \times (n-1)}$, kde

$$x_{ij} = \begin{cases} 1 & \text{holub } i \text{ je v díře } j, \\ 0 & \text{jinak.} \end{cases}$$

Lze-li přiřadit všechny holuby do děr tak, že v žádné nejsou dva, je na každém řádku je aspoň jedna hodnota 1, a v každém sloupci je nejvýše jedna 1.

Pro $i = 1, \dots, n$ uvážíme formuli $P_i = \bigvee_{j=1}^{n-1} x_{ij}$.

Pro $j = 1, \dots, n - 1$ uvážíme formuli $H_j = \bigwedge_{i, i' \in [n]; i \neq i'} (\neg x_{ij} \vee \neg x_{i'j})$.

Formule $PH_n = (\bigwedge_{i=1}^n P_i) \wedge (\bigwedge_{j=1}^{n-1} H_j)$ je očividně kontradikce, protože její platnost by popírala Dirichletův princip.

Přiřazením jednoho holuba do jedné díry dostaneme problém s o jedna méně holuby (odebereme jednoho holuba a jednu díru). Formálně definujeme ohodnocení $SET(x_{ij} = 1)$ dané $x_{ij} = 1$ a $x_{i'j} = 0$ a $x_{ij'} = 0$ pro $i' \neq i$ a $j' \neq j$. Po případném přecíslování indexů mám $PH_n[SET(x_{ij} = 1)] = PH_{n-1}$.

def. Ohodnocení α je *k-kritické*, pokud $P_k[\alpha] = 0$, ale $(PH_n - P_k)[\alpha] = 1$. (Intuitivně: v každé díře právě jeden holub a holub k není v žádné díře.)

Pozorování. Nechť α je *k-kritické* ohodnocení. Pokud $k' \neq k$ a pro každé $j \in [n - 1]$ prohodíme ohodnocení x_{kj} a $x_{k'j}$, dostaneme k' -kritické ohodnocení $\alpha' = \text{SWITCH}(\alpha, k, k')$. Přitom existuje $l \in [n - 1]$ tak, že $x_{k'l}[\alpha] = 1$, $x_{k'l}[\alpha'] = 0$, $x_{kl}[\alpha] = 0$ a $x_{kl}[\alpha'] = 1$. (Intuice: v matici prohodíme řádky k a k' . Potom musí existovat sloupec l , ve kterém jsme na místo 0 dali 1.)

def. Nechť C je klausule. Potom C^+ je klausule, kterou dostaneme nahrazením každého negativního literálu $\neg x_{ij}$ pomocí (všech) literálů $x_{i'j}$ pro $i \neq i'$. (C^+ budeme říkat *rozšířená klausule*, nebo *rozšíření* C .)

Lemma 6. Pro *k-kritické* ohodnocení α a libovolnou klauzuli C platí, že $C[\alpha] = 1$ právě, když $C^+(\alpha) = 1$.

Důkaz. Označme $\mathbf{X}_{ij} = \bigcup \{x_{i'j} \mid i' \neq i\}$. Předpokládejme $\neg x_{ij} \in C$.

Pokud $C[\alpha] = 0$, máme $(\neg x_{ij})[\alpha] = 0$ a tedy $\alpha(x_{ij}) = 1$, odtud $\mathbf{X}_{ij}[\alpha] = 0$. (Důsledek pravdivosti formule H_j z definice PH_n .)

Pokud $C[\alpha] = 1$ a $(\neg x_{ij})[\alpha] = 1$ máme $\alpha(x_{ij}) = 0$, ale $\mathbf{X}_{ij}[\alpha] = 1$ (Důsledek toho, že α je kritické.) $\square$

Věta 5. Každé rezoluční zamítnutí PH_n obsahuje klauzuli C tak, že $|C^+| > \frac{2n^2}{9}$.

Důkaz. Pro klauzuli C zvážíme množinu

$$\text{BAD}(C) = \{k \mid \text{existuje } k\text{-kritické ohodnocení } \alpha \text{ a } C[\alpha] = 0\}.$$

Máme $|\text{BAD}(P_i)| = 1$ pro každé i ; dále $|\text{BAD}(D)| = 0$ pro každou klauzuli D v H_j a každé j ; a nakonec $|\text{BAD}(\diamond)| = n$. Z korektnosti rezolučního kroku plyne

$$|\text{BAD}(r(y, C_1, C_2))| \leq |\text{BAD}(C_1)| + |\text{BAD}(C_2)|.$$

V každém rezolučním zamítnutí PH_n tak existuje klauzule C tak, že

$$\frac{n}{3} < |\text{BAD}(C)| \leq \frac{2n}{3}.$$

Vybereme $i \in \text{BAD}(C)$ a $j \notin \text{BAD}(C)$ a i -kritické ohodnocení α tak, že $C[\alpha] = 0$. Potom pro $\alpha' = \text{SWITCH}(\alpha, i, j)$ dostaneme $C[\alpha'] = 1$. Protože C^+ obsahuje pouze pozitivní literály, musí existovat právě jedna proměnná, která je α' ohodnocena nově na 1; je to x_{ik} , kde $k \in [n-1]$ je takové, že $\alpha(x_{jk}) = 1$. Víme tak, že $x_{jk} \in C^+$.

Konstrukci z předchozího odstavce zopakujeme pro všechny dvojice (i, j) , kde $i \in \text{BAD}(C)$ a $j \notin \text{BAD}(C)$, vždycky dostaneme unikátní proměnnou, která je v C^+ (pro fixní i a měnící se j dostáváme různá x_{ik} , α je totiž kritické ohodnocení). Máme tak $|C^+| \geq |\text{BAD}(C)|(n - |\text{BAD}(C)|) \geq \frac{2n^2}{9}$. $\square$

Věta 6. *Délka rezolučního zamítnutí PH_n je alespoň exponenciálně dlouhá vzhledem k n .*

Důkaz. Uvažme rezoluční zamítnutí R . Klauzuli $C \in R$ považujeme za velkou, pokud $|C^+| \geq \frac{n^2}{10}$.

Předpokládejme, že R obsahuje S velkých klauzulí (S je funkce závislá na n). Rozšíření těchto S klauzulí dohromady obsahuje aspoň $S \frac{n^2}{10}$ literálů, tj. existuje proměnná, která se vyskytuje alespoň v $\frac{S}{10}$ z nich. Označme ji x_{ij} .

Nechť $\alpha = \text{SET}(x_{ij} = 1)$. Podle Lemma existuje rezoluční zamítnutí R' formule $\text{PH}_n[\alpha] = \text{PH}_{n-1}$, které obsahuje nejvýše $\frac{2S}{10}$ velkých klauzulí (zbytek odpadne ohodnocením x_{ij}).

Předchozí krok zopakujeme, dokud nedostaneme formuli $\text{PH}_{n'}$ bez velkých klauzulí a její rezoluční zamítnutí R'' . Máme $n' \leq n - \log_{10/9} S$. Podle předchozí věty musí platit

$$\frac{2n'^2}{9} < \frac{n^2}{10},$$

po dosazení pak dostaneme

$$\frac{2(n - \log_{10/9} S)^2}{9} < \frac{n^2}{10}$$

Po technických úpravách dostaneme, že S je exponenciální funkce. $\square$

9 Devátý seminář + zbytek

Jedná se o hrubý, dosud neučesaný text

Kombinatorika

Věta 7. *Pokud každá klausule formule F obsahuje pozitivní literál, nebo každá klauzule obsahuje negativní literál, je F splnitelná.*

Důkaz. Formuli lze splnit ohodnocením všech proměnných na 0, nebo ohodnocením všech proměnných na 1. $\square$

Věta 8. *Pokud má každá klausule formule F právě k literálů a $|F| < 2^k$, pak je F splnitelná.*

Důkaz. Předpokládejme, že F neobsahuje tautologické klauzule. Nechť $n = |\text{VARS}(F)|$. Potom máme $n \leq k$. Uvažujeme ohodnocení, které ohodnotí právě proměnné z $\text{VARS}(F)$.

Pro $C \in F$ existuje 2^{n-k} ohodnocení α , pro které $C[\alpha] = 0$: literály v C ohodnotíme na 0, zbytek proměnných (těch je $n - k$) můžeme ohodnotit libovolně.

Počet ohodnocení, kterou nesplní aspoň jednu klauzuli je tak seshora omezen

$$|F| \cdot 2^{n-k} < 2^k \cdot 2^{n-k} = 2^n.$$

To znamená, že existuje ohodnocení, které splní všechny klauzule. $\square$

Věta 9 (Obecnější verze). *Nechť $F = \{C_1, C_2, \dots, C_m\}$. Pokud $\sum_{j=1}^m 2^{-|C_j|} < 1$, je F splnitelná.*

Důkaz. Analogie důkazu předchozí věty.

Pro $C \in F$ existuje $2^{-|C|} \cdot 2^n$ ohodnocení, které nesplní C . Součtem přes všechny klauzule máme

$$\left(\sum_{j=1}^m 2^{-|C_j|} \right) \cdot 2^n < 1 \cdot 2^n = 2^n.$$

To znamená, že existuje ohodnocení, které splní všechny klauzule. $\square$

Důkaz. (Alternativní)

K $F = \{C_1, C_2, \dots, C_m\}$ s $\sum_{j=1}^m 2^{-|C_j|} < 1$ najdeme α tak, že $F[\alpha] = 1$.

Pro literál l definujeme číslo $S_l = \sum_{C \in A} 2^{-|C|}$, kde A je množina všech klauzulí z F , které obsahují literál l .

Předpokládáme, že F neobsahuje triviálně pravdivé klauzule. Vybereme proměnnou $x \in F$. Nechť $S = \sum_{C \in A} 2^{-|C|}$, kde A je množina klauzulí, které neobsahují literál proměnné x .

Hned vidíme, že

$$S_x + S_{\neg x} + S = \sum_{j=1}^m 2^{-|C_j|}.$$

Máme tak $S_x + S_{\neg x} + S < 1$.

Dále máme

$$\sum_{C \in F[\{x=0\}]} = 2S_x + S \tag{1}$$

Při přechodu od F k $F[\{x=0\}]$ totiž odstraníme klauzule obsahující $\neg x$. Dále z klauzulí obsahujících x tento literál odstraníme a zmenšíme tak jejich velikost o 1.

Analogicky dostaneme

$$\sum_{C \in F[\{x=1\}]} = 2S_{\neg x} + S \tag{2}$$

Pokud sečteme pravé strany, dostaneme $2S_x + 2S_{\neg x} + 2S$. Tato hodnota musí být ostře menší než 2 (protože $2S_x + 2S_{\neg x} + 2S \geq 2$ implikuje $S_x + S_{\neg x} + S \geq 1$). Tedy aspoň jeden výraz z (1), (2) musí být menší než 1. Proměnou x na a , kde $a = 0$, pokud je menší (1), jinak je $a = 1$. Klauzule z množiny $F - F[\{x = a\}]$ jsou obsahují pravdivý literál proměnné x , a samotná formule $F[\{x = a\}]$ opět splňuje náš předpoklad a aplikujeme na ni výše uvedený algoritmus. $\square$

def. Pro klauzuli C formule F definujeme množinu jejích sousedů

$$\eta(C) = \{C' \in F \mid \text{vars}(C) \cap \text{vars}(C') \neq \emptyset\}.$$

Věta 10 (Lokální Lovácsovo lemma). *Nechť všechny klauzule ve formuli F mají velikost právě k . Pokud pro každou klauzuli C formule F platí $|\eta(C)| < 2^{k-2}$, je F splnitelná.*

Důkaz. (vynecháme některé technické detaily, lze brát jako kostru důkazu.)

V důkazu využijeme následující tvrzení:

Náhodný řetězec bitů s velkou pravděpodobností nelze zkomprimovat.

Toto tvrzení můžeme chápat následovně. Pro každou dvojici f, g algoritmů (vstup i výstup jsou bitové řetězce) takových, že pro každý bitový řetězec s máme $g(f(s)) = s$, platí, že pro náhodně vybraný dlouhý bitový řetězec s' je s velkou pravděpodobností $|f(s')| \geq |s'|$.

Nechť $F = \{C_1, C_2, \dots, C_m\}$. Uvážíme následující procedury, α je globální proměnná.

MAIN

1. V rámci α náhodně ohodnot' proměnné z $\text{VARS}(F)$.
2. Pro $j = 1, 2, \dots, m$: pokud $C_j[\alpha] = 0$, spust' $\text{REPAIR}(C_j)$.

REPAIR(C)

1. V rámci α náhodně ohodnot' proměnné z $\text{VARS}(C)$.
2. Pro $j = 1, 2, \dots, m$: pokud $C_j \in \eta(C)$ a $C_j[\alpha] = 0$, spust' $\text{REPAIR}(C_j)$.

Nejdříve si dokážeme následující pomocné tvrzení.

Pomocné tvrzení. *Pokud volání $\text{REPAIR}(C)$ skončí, potom*

- $C[\alpha] = 1$,
- Všechny klauzule C' , které byli pravdivé pro α před voláním, jsou pravdivé i pro α po volání.

Důkaz. Indukcí přes výšku stromu rekurzivních volání.

Pokud je výška 0, ve volání REPAIR v kořeni nedojde k dalšímu rekurzivnímu volání. Řádek 1 může zneplatnit pouze klauzule z množiny $\eta(C)$. Pro ně ovšem na řádce 2 nedošlo k rekurzivnímu volání a musí tedy být pravdivé.

Předpokládejme, že tvrzení platí pro volání s rekurzivním stromem výšky h . Tvrzení dokážeme pro strom výšky $h + 1$. V kořeni tohoto stromu došlo k rekurzivnímu volání na řádce 2, ovšem jejich podstrom má výšku h . Použijeme indukční předpoklad a vidíme, že klauzule z množiny $\eta(C)$ jsou pravdivé. ■

Pokud tedy procedura MAIN skončí, je $F[\alpha] = 1$. Ve zbytku důkazu ukážeme, že MAIN skončí s nenulovou pravděpodobností.

Představíme si výpočet, při kterém došlo k s zavoláním REPAIR . Tento výpočet potřeboval $sk + n$ náhodných bitů: každé volání REPAIR potřebuje k náhodných bitů, volání MAIN potřebuje n náhodných bitů. Níže ukážeme, že výpočet umožňuje těchto $sk + n$ bitů zkomprimovat na $n + m + s(k - \epsilon)$ bitů, ze kterých lze původní bity zrekonstruovat (ϵ je kladná konstanta). S použitím předpokladu ze začátku důkazu tak s velkou pravděpodobností máme

$$n + sk \leq n + m + s(k - \epsilon)$$

a tedy $s \leq \frac{m}{\epsilon}$. Výpočet tak s velkou pravděpodobností skončí po $\frac{m}{\epsilon}$ voláních REPAIR .

Princip komprese: při výpočtu si zapamatujeme následující.

- Ohodnocení α na konci výpočtu (n bitů)
- Klausule, pro které došlo v MAIN k volání REPAIR (m bitů)
- Pro každé volání REPAIR si pamatujeme klauzule, u kterých došlo k rekurzivním voláním. Protože může dojít k maximálně $2^{k-2-\epsilon}$ těmto voláním ($|\eta(C)| < 2^{k-2}$), stačí nám $k-2-\epsilon$ bitů.
- Pomocí záznamu průchodu do hloubky z kořene si zapamatujeme tvar stromu rekurze. Tento záznam má $2s$ bitů, 1 značí přechod do prvního dosud nenavštíveného potomka, 0 značí přechod k rodiči.

Celkem je tedy potřeba $n + m + s(k-2-\epsilon) + 2s = n + m + s(k-\epsilon)$ bitů.

Princip dekomprese. Uvážíme posloupnost

$$\alpha_0, z_0, \alpha_1, z_1, \dots, z_{s-1}, \alpha_s,$$

kde α_i jsou ohodnocení proměnných, z_i je argument REPAIR v i -tém zavolání (bránu podle času, kdy volání provedla krok 1.) Ohodnocení α_i je hodnota α poté, co $i-1$ volání REPAIR provedlo krok 1. (α_0 nastavila v kroku 1 procedura MAIN.)

Díky tomu, co jsme si zapamatovali při kompresi můžeme zjistit hodnoty z_0 až z_s . Důvod: rekurzivní volání jsou seřazena v pořadí průchodu stromem do hloubky, jehož tvar známe. Známe-li argument REPAIR můžeme zjistit argumenty jeho rekurzivních zavolání (protože známe množinu η).

Známe také α_s . Pokud v posloupnosti $\alpha_i z_i \alpha_{i+1}$ známe α_{i+1} a z_i , můžeme jednoznačně určit α_i . Ohodnocení α_i se liší od α_{i+1} pouze v proměnných obsažených v klauzuli z_i . Tyto proměnné ovšem musí být v klauzuli z_i nastaveny tak, aby z_i byla nepravdivá, protože pouze v tomto případě došlo k volání REPAIR pro z_i . Všechny náhodné bity použité ve výpočtu jsme tedy schopni získat. $\square$

Věta 11 (LLL, obecnější varianta). *Pokud pro každou $C \in F$ platí*

$$\sum_{D \in \eta(c)} 2^{-|D|} < \frac{1}{4},$$

je F splnitelná.

V následujícím uvažujeme pouze neorientované grafy.

def. *Párování* v grafu $G = (V, H)$ je množina hran $H' \subseteq H$ taková, že žádné dvě její hrany nesdílí vrchol. Párování H' pokrývá množinu vrcholů $V' \subseteq V$, pokud každý vrchol z V' inciduje s nějakou hranou z H' .

def. Graf $G = (V, H)$ je *bipartitní*, pokud existují $V_1, V_2 \subseteq V$ tak, že $V = V_1 \cup V_2$ a $V_1 \cap V_2 = \emptyset$ a každá hrana grafu inciduje z jedním vrcholem z V_1 a jedním vrcholem z V_2 . Množinám V_1 a V_2 říkáme *partity*.

def. Nechť $G = (V, H)$ je graf. Potom pro $v \in V$ definujeme množinu vrcholů $N_G(v) = \{v' \in V \mid \{v, v'\} \in H\}$.

Věta 12 (Hallův marriage theorem). *Nechť $G = (V, H)$ je bipartitní graf s partitami A, B . Potom existuje párování, které pokrývá A , právě když pro každé $S \subseteq A$ platí, že $|N_G(s)| \geq |S|$.*

Důkaz.

existence párování implikuje podmínku:

Nechť $S \subseteq A$. Potom množina $\{v \in B \mid v' \in A, \{v, v'\} \text{ je v párování} \}$ obsahuje právě $|S|$ prvků, protože párování pokrývá A .

podmínka implikuje existenci párování:

Důkaz je indukcí přes $|A|$. Pokud $|A| = 1$, pak z podmínky plyne $|N(A)| \geq 1$ a existuje tak hrana (a, b) pro $a \in A, b \in N(A)$ tvořící párování.

Předpokládejme $|A| \geq 2$ a že pro menší partity tvrzení platí. Rozlišíme dva případy.

1. Pro každou neprázdnou $S \subset A$ máme $|N_G(S)| \geq |S| + 1$.

Vybereme hranu $\{a, b\}$, kde $a \in A, b \in B$. Uvážíme podgraf G' grafu G indukovaný množinou $V - \{a, b\}$. Potom pro každou neprázdnou $S \subseteq A - \{a\}$ máme $|N_{G'}(S)| \geq |N_G(S)| - 1 \geq |S|$, protože odebráním vrcholu b z grafu ztratí libovolný vrchol $z \in A$ nejvýše jednoho souseda. Na G' a partitu $A - \{a\}$ aplikujeme indukční předpoklad a dostaneme párování. Když k němu přidáme hranu $\{a, b\}$, dostaneme párování pokrývající A .

2. Existuje neprázdné $A' \subset A$ tak, $|A'| = |N_G(A')|$.

Označme $B' = N_G(A')$. Nechť G' je podgraf indukovaný množinou $A' \cup B'$. Potom G' splňuje podmínku a můžeme na něj uplatnit indukční předpoklad: existuje párování P_1 , které pokrývá A' .

Nechť G'' je podgraf indukovaný množinou $V - (A' \cup B')$. Pokud by existovala $S \subseteq A - A'$ taková, že $N_{G''}(S) < |S|$, potom bychom měli $|N_G(S \cup A')| < |S \cup A'|$, co je porušení podmínky pro G . Pro G'' tedy podmínka také platí a z indukčního předpokladu plyne existence párování P_2 , které pokrývá $A - A'$.

Množina $P_1 \cup P_2$ je tak párování, které pokrývá A .

□

Věta 13. Nechť F je formule. Pokud pro každé $G \subseteq F$ platí $|G| \leq |\text{vars}(G)|$, je F splnitelná.

Důkaz. Sestavíme bipartitní graf s partitami F a $\text{vars}(F)$ a množinou hran $\{\{C, x\} \mid x \in \text{vars}(C)\}$. Jedná se o bipartitní graf, ve kterém podle předchozí věty existuje bipartitní párování pokrývající F . Toto párování každé klausuli z F přiřazuje jinou proměnnou. Vhodným ohodnocením této proměnné danou klauzulí splníme. □

Věta 14. Nechť F je formule, kde má každá klausule nejméně k literálů. Pokud má každá proměnná literál v nejvýše k klauzulích, je F splnitelná.

Důkaz. Vybereme libovolné $G \subseteq F$. Potom G obsahuje nejméně $k|G|$ výskytů literálů, protože součet velikostí klausulí v G je nejméně $k|G|$. Přitom každá proměnná má literál v nejvýše k klauzulích. Musí tedy platit $|G| \leq |\text{Var}(G)|$ a tvrzení plyne z předchozí věty. □

Pokud se ve formuli F vyskytuje každá proměnná v nejvýše dvou klauzulích, lze o její splnitelnosti rozhodnout v polynomičtém čase následujícím algoritmem.

1. Dokud existuje $\{l\} \in F$ nastavíme $F \leftarrow F \setminus \{\{l\}\}$.
2. Pokud $\diamond \in F$, není F splnitelná, jinak splnitelná je.

Algoritmus je korektní, protože pokud v bodu 2 $\diamond \notin F$, obsahuje F pouze klauzule s alespoň dvěma literály a podle předchozí věty je splnitelná.

Formuli, ve které se některá proměnná vyskytuje v $s > 3$ klauzulích, lze v polynomičtém čase převést na formuli F' , kde se proměnné vyskytují v nejvýše 3 klauzulích a F je splnitelná právě když je F' splnitelná. Za předpokladu $P \neq NP$ tak analogie algoritmu výše nemůže fungovat pro

formule, kde se proměnné vyskytují v nejvýše 3 klauzulích.

Princip převodu je následující: s výskytů literálu proměnné x nahradíme postupně literály nových proměnných $x_1, x_2, \dots, x_s$, zachováme při tom negace (tj. pozitivní literál x nahradíme pozitivním literálem, negativní literál x negativním literálem). K F také přidáme formuli

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge \dots \wedge (\neg x_s \vee x_1).$$

Tato formule je pravdivá pouze pro ohodnocení α , pro které $\alpha(x_i) = \alpha(x_j)$ pro všechna $i, j \in \{1, 2, \dots, s\}$.

Věta 15. *Nechť F je kontradikce taková, že pro každé $C \in F$ je formule $F - C$ splnitelná. Potom $|F| > |\text{Var}(F)|$.*

Důkaz. Z Věty 13 víme, že pokud je F kontradikce, pak existuje $G \subseteq F$ tak, že $|G| > |\text{Vars}(G)|$. Předpokládejme, že G má mezi takovými množinami maximální velikost. Ukážeme, že $F = G$. Ukážeme, že $G \neq F$ implikuje, že F je splnitelná a to je spor.

Předpokládejme $G \neq F$. Pro každou neprázdnou množinu $H \subseteq F - G$ platí

$$|H| < |\text{Vars}(H) - \text{Vars}(G)| \leq |\text{Vars}(H)|.$$

Kdybychom totiž měli $|H| \geq |\text{Vars}(H) - \text{Vars}(G)|$, platilo by $|G \cup H| > |\text{Vars}(G)| + |\text{Vars}(H)| \leq |\text{Vars}(G \cup H)|$ a G by neměla maximální velikost.

Formule $F - G$ je tedy podle Věty 13 splnitelná, navíc stačí ohodnoti proměnné z množiny $\text{Vars}(F) - \text{Vars}(G)$ (viz důkaz Věty 13).

Z předpokladu o F plyne, že G je splnitelná. Tím pádem je ovšem formule $F = (F - G) \cup G$ také splnitelná: G splníme ohodnocením proměnných ve $\text{Vars}(G)$ a $F - G$ pomocí proměnných ve $\text{Vars}(F) - \text{Vars}(G)$. To je ovšem spor, protože F je kontradikce. Musí tedy platit $F = G$. $\square$